LÖSUNGEN · LEHREREXEMPLAR

W2·2 – Sortieren: Lösungen

Zum Arbeitsblatt `AQ_W2_AB2_Sortieren.docx` · IDE-Level 2.2: siehe `AQ_W2_L2_Rangliste_LSG.txt` · alle Zustände und Zählerstände mit Java nachgerechnet

Aufgabe 0 – Sortieren ohne Computer

Erwartbar: Varianten von Selectionsort („größte Karte suchen und nach vorne"), Insertionsort („wie beim Kartenspielen einsortieren" – mit der Tausch-Regel nur über Nachbartausche) und Bubblesort („Nachbarn vergleichen"). Orientierung bei 8 Karten: Selectionsort braucht immer 7 + 6 + … + 1 = **28** Vergleiche, Insertionsort zwischen **7** (schon sortiert) und 28, Bubblesort mit Abbruch zwischen 7 und 28. Gute Sicherungsfrage zu d): Wer weniger Vergleiche hatte, hatte oft „günstigere" Karten – Aufwand hängt von Verfahren **und** Eingabe ab (→ W2·3: bester/schlechtester Fall).

Aufgabe 1 – Dreieckstausch

nach Zeile	hilf	eintraege[1]	eintraege[3]
Eintrag hilf = eintraege[i];	Brann	Brann	Dorn
eintraege[i] = eintraege[j];	Brann	Dorn	Dorn
eintraege[j] = hilf;	Brann	Dorn	Brann

b) Danach steht **Aria, Dorn, Cyra, Dorn** in der Liste. Die erste Zeile überschreibt die einzige Referenz auf Brann; die zweite kopiert nur Dorn zurück. Auf das Brann-Objekt verweist nichts mehr – der Eintrag ist verloren (und wird vom Garbage Collector entfernt).

Aufgabe 2 – Trace für 50, 80, 30, 90, 70 (12 BE)

a) Selectionsort:

nach …	Index 0	Index 1	Index 2	Index 3	Index 4	Vergleiche	Vertauschungen
Start	50	80	30	90	70	0	0
i = 0	90	80	30	50	70	4	1
i = 1	90	80	30	50	70	7	1
i = 2	90	80	70	50	30	9	2
i = 3	90	80	70	50	30	**10**	**2**

b) Insertionsort:

nach …	Index 0	Index 1	Index 2	Index 3	Index 4	Vergleiche	Vertauschungen
Start	50	80	30	90	70	0	0
i = 1	80	50	30	90	70	1	1
i = 2	80	50	30	90	70	2	1
i = 3	90	80	50	30	70	5	4
i = 4	90	80	70	50	30	**8**	**6**

Bei i = 3 wandert 90 bis ganz nach vorne: 3 Vergleiche, dann ist j = 0 und die Schleife endet **ohne** weiteren Vergleich (Kurzschluss). Bei i = 4: 70 > 30, 70 > 50, 70 > 80 nein → 3 Vergleiche, 2 Tausche.

c) Bubblesort mit Abbruch:

nach …	Index 0	Index 1	Index 2	Index 3	Index 4	Vergleiche	Vertauschungen
Start	50	80	30	90	70	0	0

nach …	Index 0	Index 1	Index 2	Index 3	Index 4	Vergleiche	Vertauschungen
Durchlauf 1	80	50	90	70	30	4	3
Durchlauf 2	80	90	70	50	30	7	5
Durchlauf 3	90	80	70	50	30	9	6
Durchlauf 4	90	80	70	50	30	**10**	**6**

Durchlauf 4 vergleicht nur noch 80 mit 90, tauscht nicht → getauscht bleibt falsch → Ende. (Ohne Abbruch wären es hier ebenfalls 10 Vergleiche – bei n = 5 läuft die verkürzte innere Schleife ohnehin nur 4 + 3 + 2 + 1-mal.)

d) Selectionsort vertauscht am wenigsten (**2**): höchstens ein Tausch pro Durchlauf, insgesamt höchstens n − 1. Insertionsort vergleicht am wenigsten (**8**): Die innere Schleife bricht ab, sobald der linke Nachbar besser ist – bereits gut stehende Einträge kosten nur einen Vergleich. Bubble- und Insertionsort vertauschen **gleich oft** (6): Beide tauschen nur Nachbarn, und jeder Nachbartausch beseitigt genau einen **Fehlstand** (Paar in falscher Reihenfolge); die Liste hat 6 Fehlstände. Selectionsort vergleicht immer n(n − 1)/2 = 10-mal, egal wie die Daten aussehen (3 BE).

Aufgabe 3 – Level 2.2

c) Demo-Liste 1200, 800, 1500, 950, 600, 1100 mit insertionSort: **10 Vergleiche, 6 Vertauschungen**, Ergebnis Cyra, Aria, Finn, Dorn, Brann, Elin (mit Java nachgerechnet). Weitere Lösungen und häufige Fehler: `AQ_W2_L2_Rangliste_LSG.txt`.

Aufgabe 4 – Stabilität (5 BE)

a) Selectionsort: **Brann, Dorn, Cyra, Aria** · Insertionsort: **Brann, Dorn, Aria, Cyra** (2 BE).

b) Insertionsort arbeitet stabil: Es tauscht nur **Nachbarn** und nur, wenn der rechte **strikt** besser ist – gleich gute Einträge überholen sich nie. Selectionsort ist nicht stabil: Im Durchlauf i = 1 ist Dorn (Index 3) der Beste des Restes und wird mit **Aria (Index 1)** vertauscht. Dieser **Fernaustausch** schiebt Aria hinter Cyra, obwohl beide 500 Punkte haben und Aria zuerst da war (3 BE).

Denkanstoß: Bubblesort ist stabil (nur Nachbartausche bei strikt besser). Mit `>=` würden gleich gute Nachbarn getauscht: Insertionsort liefert dann Dorn, Brann, Cyra, Aria – die Reihenfolge Gleicher wird umgedreht (nicht stabil) und es gibt 5 statt 3 Vertauschungen (mit Java geprüft). Anwendung: Sortiert man zuerst nach Spielzeit und danach **stabil** nach Punkten, steht bei Punktgleichstand automatisch der Schnellere vorne.

Aufgabe 5 – Abi-Training: Verfahren erkennen (11 BE)

nach …	r[0]	r[1]	r[2]	r[3]	r[4]
i = 0	1	7	3	9	5
i = 1	1	3	7	5	9
i = 2	1	3	5	7	9
i = 3	1	3	5	7	9

a) siehe Tabelle (3 BE; je falscher Zeile 1 BE Abzug, Folgefehler berücksichtigen).

b) Erwartungshorizont (5 BE): Es handelt sich um **Bubblesort** (1). Die innere Schleife läuft mit j von rechts (Länge von r − 1) nach links bis i + 1 und vergleicht immer zwei **benachbarte** Elemente r[j − 1] und r[j]; steht das kleinere rechts, werden beide mit Hilfsvariable vertauscht (2). Dadurch wandert das kleinste Element des noch unsortierten Bereichs i … Länge − 1 nach links an die Position i („steigt auf“) (1). Die äußere Schleife zählt die Durchläufe: Nach dem Durchlauf mit i stehen die i + 1 kleinsten Elemente endgültig sortiert vorne; der zu bearbeitende Bereich wird daher bei jedem Durchlauf um eins kleiner, n − 1 Durchläufe genügen. Ergebnis: aufsteigend sortiert (1). **Abgrenzung:** kein Selectionsort – es wird kein Minimum gesucht und weit getauscht, sondern nur benachbart; kein Insertionsort – das bewegte Element kommt aus dem unsortierten Teil von rechts, und die Schleife bricht nicht vorzeitig ab.

c) Die Bedingung wird 4 + 3 + 2 + 1 = **10**-mal geprüft (unabhängig von den Werten), es finden **6** Vertauschungen statt (3 + 2 + 1) (2 BE). **d)** Bedingung zu `r[j] > r[j − 1]` ändern → 9, 7, 5, 3, 1 (1 BE).

Aufgabe 6 – Sortiert einfügen

a) Cyra, Aria, Dorn, **Hugo**, Brann – **2 Vergleiche**: Hugo ist besser als Brann (Brann wird geschoben), aber nicht besser als Dorn (950 = 950, besser ist strikt) → Hugo kommt hinter Dorn. Das hält die Liste stabil: Dorn war zuerst da (2 BE).

```
public boolean einfuegenSortiert(Eintrag e) {
    if (e == null || anzahl >= eintraege.length) {
        return false;
    }
    int i = anzahl;
    while (i > 0 && besser(e, eintraege[i - 1])) {
        eintraege[i] = eintraege[i - 1];
        i--;
    }
    eintraege[i] = e;
    anzahl++;
    return true;
}
```

b) Lücken (4 BE, je Zeile ½ BE, zusammen gerundet): `e == null || anzahl >= eintraege.length` (auch `== eintraege.length`) · `anzahl` · `besser(e, eintraege[i - 1])` · `eintraege[i - 1]` · `eintraege[i]` · `anzahl++`. **Typische Fehler:** `besser(eintraege[i - 1], e)` (falsche Richtung) · `i > 0` hinter dem Vergleich · Schieben nach links (`eintraege[i - 1] = eintraege[i]`) · anzahl nicht erhöht · Prüfung auf voll fehlt → ArrayIndexOutOfBoundsException.

d) Nach jedem Einfügen ist die Liste wieder sortiert (die leere und die einelementige Liste sind es ohnehin). Nach dem letzten Einfügen sind also alle fünf Einträge sortiert. Das ist genau **Insertionsort** – nur mit Verschieben statt Nachbartausch (2 BE). Der Test `experte_volleListeUndEinfuegenGleichSortieren` prüft das.

Für Schnelle – BogoSort

5 verschiedene Einträge: 5! = **120** Reihenfolgen; genau eine ist sortiert → im Mittel etwa **120** Mischversuche. 10 Einträge: 10! = **3 628 800**. 20 Einträge: 20! ≈ **$2{,}4 \cdot 10^{18}$** Versuche; bei 10^8 Versuchen pro Sekunde ≈ $2{,}4 \cdot 10^{10}$ s ≈ **770 Jahre** (und jeder Versuch kostet zusätzlich bis zu n − 1 Vergleiche in istSortiert). Hinweis für die Lehrkraft: Das Mischen mit `vertausche(i, zufall aus 0 … anzahl − 1)` erzeugt nicht alle Reihenfolgen exakt gleich wahrscheinlich (korrekt wäre zufall aus i … anzahl − 1, Fisher-Yates) – für die Größenordnung egal, schöner Anlass für Experten.

```
public void bogoSort() {
    int versuche = 0;
    while (!istSortiert()) {
        for (int i = 0; i < anzahl; i++) {
            vertausche(i, (int) (Math.random() * anzahl));
        }
        versuche++;
    }
    println("Versuche: " + versuche);
}

public void sortiereNachName() {           // Insertionsort, alphabetisch
    for (int i = 1; i < anzahl; i++) {
        int j = i;
        while (j > 0 && name(j).compareTo(name(j - 1)) < 0) {
            vertausche(j, j - 1);
            j--;
        }
    }
}

private String name(int i) {                // Hilfsmethode: kuerzere Zeilen
    return eintraege[i].getName();
}
```

`<` ist in Java nur für Zahlen (und Zeichen) definiert; Zeichenketten sind Objekte. Der lexikographische Vergleich `compareTo` liefert eine negative Zahl, wenn die erste Zeichenkette alphabetisch vorher kommt (erlaubte Operation laut EH). Achtung: Die Demo-Namen Aria … Finn sind schon alphabetisch – zum Testen vorher nach Punkten sortieren.